CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

The Automating Data Migration of Student Records system is designed to streamline and automate the process of transferring student data from Excel files to a MySQL database. The project eliminates manual data entry, reduces errors, and improves efficiency in handling large datasets.

This system integrates Python, Selenium, MySQL, PHP, and HTML to provide a fully automated workflow for student record management. It extracts student details from Excel, validates the data, inserts it into a MySQL database, and automates web form submissions using Selenium.

1.2 GENERAL ORGANIZATION OF THE REPORT

This report on Automating Data Migration of Student Records is structured to provide clear, detailed, and technical insights into the development, implementation, and testing of the system. The report is designed for readers such as students, faculty members, developers, and system administrators who want to understand the technical workings, system design, and testing methodologies of the project.

HOW TO USE THIS REPORT

This document provides detailed insights into the planning, design, development, testing, and deployment of the Automating Data Migration of Student Records project. Readers can use it for the following purposes:

- Understanding the problem statement and the need for automation.
- Learning the system architecture and how different components work together.

- Exploring the technologies used, including Python, MySQL, Selenium, and PHP. Following step-by-step testing and validation to ensure system efficiency.
- Deploying the system successfully in a real-world environment.

The report is structured into eight chapters, allowing readers to easily navigate and focus on specific sections based on their interests.

STRUCTURE OF THE REPORT

Chapter	**Description**
Chapter 1: Introduction	Provides an overview of the project, its importance, objectives, and scope.
Chapter 2: System Analysis	Discusses problems in the existing system and how the proposed system solves them.
Chapter 3: System Design	Explains the architecture, data flow, and database design, supported by diagrams.
Chapter 4: Project Description	Details the functional modules, key components, and automation process.
Chapter 5: System Development	Covers programming languages, frameworks, tools, and database setup.
Chapter 6: System Testing and Validation	Describes the testing process, validation techniques, and performance analysis.
Chapter 7: User Manual	Provides a step-by-step guide on how to use the system, including troubleshooting.
Chapter 8: System Deployment	Outlines the installation, configuration, execution, and security considerations.

CHAPTER 2

SYSTEM ANALYSIS

2.1 Problem Definition:

The current manual process of managing student details in Excel spreadsheets is inefficient, error-prone, and lacks scalability. As the volume of student data grows, the system becomes increasingly difficult to maintain, leading to data inconsistencies, human errors, and challenges in generating reports or conducting meaningful analysis. Additionally, the manual workflow increases the risk of data loss and makes collaboration between departments cumbersome. To address these issues, there is a need for an automated solution that ensures accurate, scalable, and efficient management of student data through migration to a structured database system.

2.2 Existing System:

Before automation, student details are manually managed in Excel spreadsheets, leading to time-consuming data entry and a high risk of human errors. The process lacks scalability, making it difficult to handle large datasets, with limited validation and inconsistent data. Generating reports or conducting advanced data analysis is inefficient due to Excel's basic functionalities. Additionally, manual processes increase the risk of data loss and make data sharing across departments cumbersome.

2.3 Proposed System:

The proposed solution is to automate the data migration process using Robotic Process Automation (RPA) with Python. This involves extracting student details from Excel, transforming the data to fit the database schema, and loading it into a structured database system. Python libraries like Pandas ensure data validation, consistency, and accuracy during the migration. This automation reduces manual effort, minimizes errors, improves scalability, and enables efficient querying and reporting of student data.

2.4 SYSTEM REQUIERMENTS

2.4.1 Hardware Requirements

Processor :Intel Core i5 or AMD Ryzen 5

RAM :4 GB

Hard Disk :160 GB

Compact Disk :650 MB

Keyboard : Standard Keyboard

2.4.2 Software Requirements

Operating System : Windows 7 or 10.

Frontend : HTML

Backend : Python

Database : MySQL Server

Tool : PyCharm

CHAPTER 3

SYSTEM DESIGN

System design is the process of defining the architecture, modules, interfaces and data for a system to satisfy system requirements.

3.1 Architectural Design

Architectural design in software engineering is about decomposing the system into interacting components. It is expressed as a block diagram defining an overview of the system structure, features of the components, and how these components communicate with each other to share data. It identifies the components that are necessary for developing a computer-based system and communication between the relationships between these components.

It defines the structure and properties of the components that are involved in the system and also the interrelationships between these components. The architectural design process is about identifying the components i.e. subsystems that makeup the system and structure of the sub-system and they're interrelationship. It is an early stage of the system design phase. It acts as a link between specification requirements and the design process.

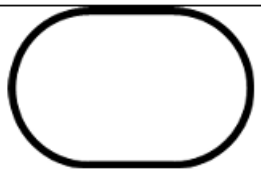
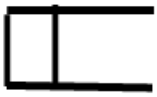
Database architecture uses programming languages to design a particular type of software for businesses or organizations. Database architecture focuses on the design, development, implementation and maintenance of computer programs that store and organize information for businesses, agencies and institutions. A database architect develops and implements software to meet the needs of users.

The design of a DBMS depends on its architecture. It can be centralized or decentralized or hierarchical. The architecture of a DBMS can be seen as either single tier or multitier. Big data architecture can handle ingestion, processing, and analysis of data that is too large or too complex for traditional database systems.

DATA FLOW DIAGRAM

A data flow diagram (DFD) is a visual representation of how information flows through a system or process. Data Flow Diagram use standardized symbols and text labels to show data inputs, outputs, storage points, and the routes between each destination. A two-dimensional diagram explains how data is processed and transferred in a system. The graphical depiction identifies each source of data and how it interacts with other data sources to reach a common output. Individuals seeking to draft a dataflow diagram must identify external inputs and outputs, determine how the inputs and outputs relate to each other, and explain with graphics how these connections relate and what they result in. This type of diagram helps business development and design teams visualize how data is processed and identify or improve certain aspects.

Data flow Symbols:

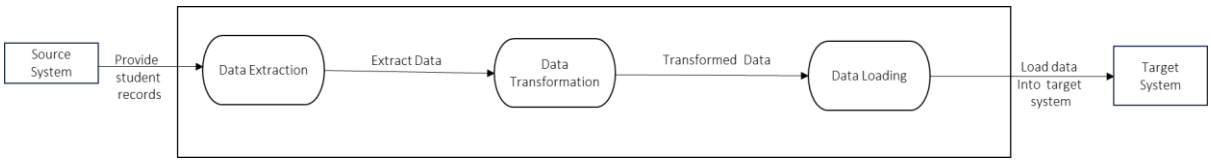
Symbols	Description
	An entity. A source of data or a destination for data.
	A process or a task that is performed by the system.
	A data store, a place where data is held between processes.
	A data flow.

LEVEL 0

The Level 0 DFD shows how the system is divided into 'sub-systems' (processes), each of which deals with one or more of the data flows to or from an external agent, and which together provide all of the functionality of the system as a whole. It also identifies internal data stores that must be present in order for the system to do its job, and shows the flow of data between the various parts of the system.

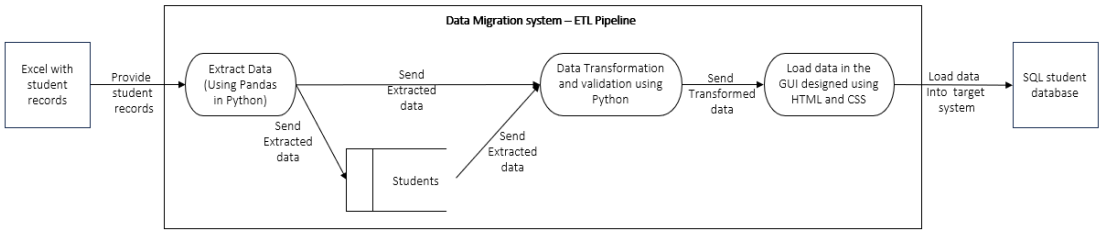
LEVEL 1

At Level 1, the Data Flow Diagram (DFD) provides a more detailed view of the system, breaking down the processes identified in Level 0 into further processes. This level illustrates the interactions between different components within each subsystem and highlights the specific data flows between processes. It delineates the functional decomposition of the system, showing how high-level processes are broken down into smaller, more manageable tasks.

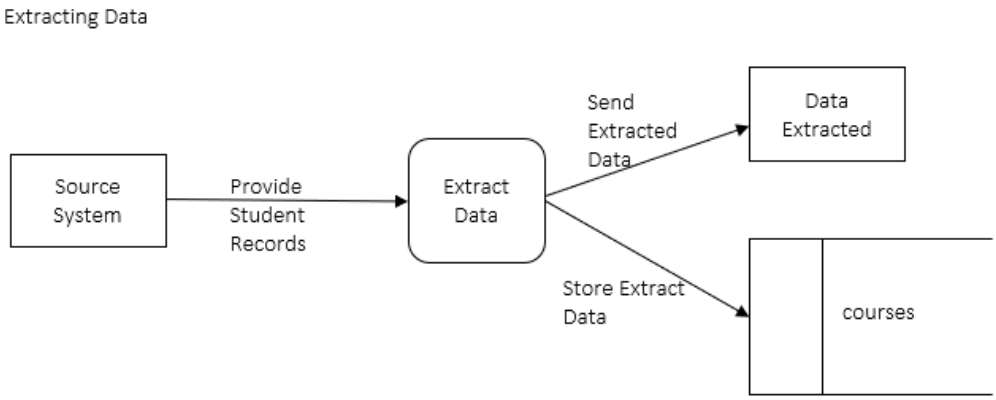


LEVEL 2

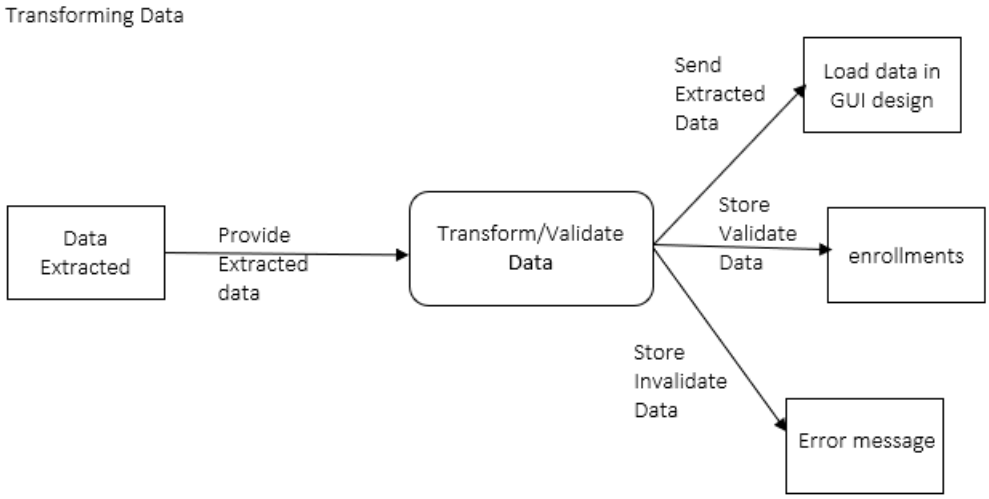
A level 2 data-flow diagram (DFD) is a decomposed level 1 DFD process that represents the processes within each function of the system and how they interact with each other. A level 1 DFD provides an overview of the system, while a level 2 DFD represents a more detailed view of the system by breaking down the sub-processes identified in the level 1 DFD into further sub-processes. Each sub-process is depicted as a separate process on the level 2 DFD, and the data flows and data stores associated with each sub-process are also shown



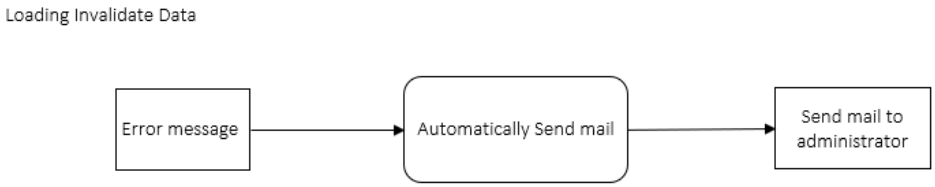
LEVEL 2.1



LEVEL 2.2



LEVEL 2.3



3.2 DATABASE DESIGN

1. Student Table

Stores details about students.

Column Name	Data Type	Constraints	Description
StudentID	INT	PRIMARY KEY	Unique ID for each student
Name	VARCHAR(100)	NOT NULL	Full name of the student
DOB	DATE	NOT NULL	Dob of the student
Gender	VARCHAR(10)	NOT NULL	Gender of the student
ContactNumber	VARCHAR(15)	UNIQUE	Contact number of student
Email	VARCHAR(100)	UNIQUE	Email ID of student
Address	VARCHAR(100)	NOT NULL	Address of the Student

2. Course Table

Stores details about courses.

Column Name	Data Type	Constraints	Description
StudentID	INT	PRIMARY KEY	Unique ID for each students
CourseID	INT	PRIMARY KEY	Unique ID for each course
Course Name	VARCHAR(100)	NOT NULL	Name of the course
Department Name	VARCHAR(100)	NOT NULL	Name of the department

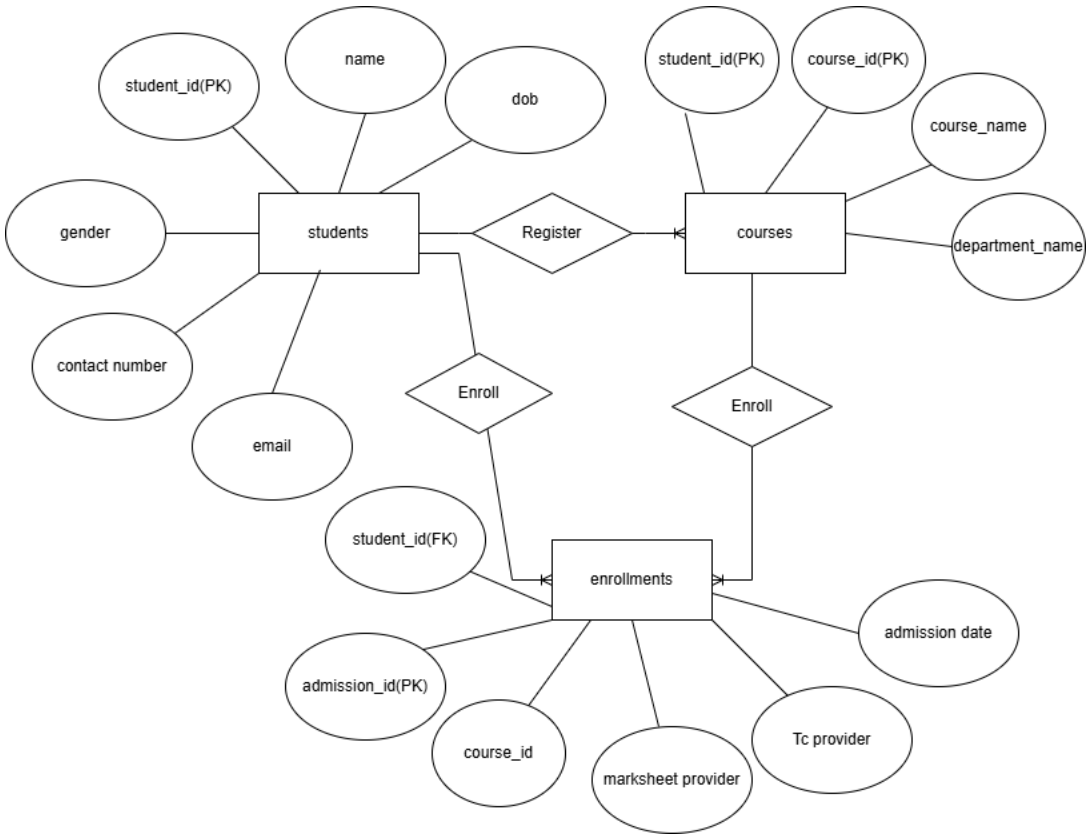
3. Admission Table

Stores information about which students are enrolled in which courses.

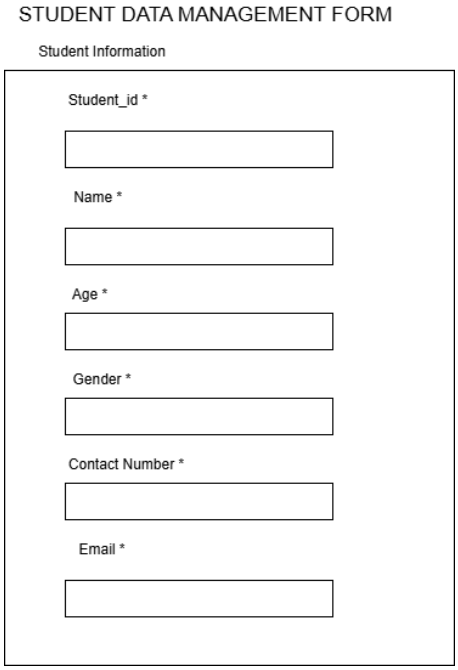
Column Name	Data Type	Constraints	Description
AdmissionID	INT	PRIMARY KEY	Unique ID for enrolment
StudentID	INT	PRIMARY KEY	References the Student table and course table
Marksheet Provided	BOOLEAN	NOT NULL	Whether the marksheet was provided by the student
TC Provided	BOOLEAN	NOT NULL	Whether the TC was provided by the student
EnrollmentDate	DATE	NOT NULL	Date of enrolment

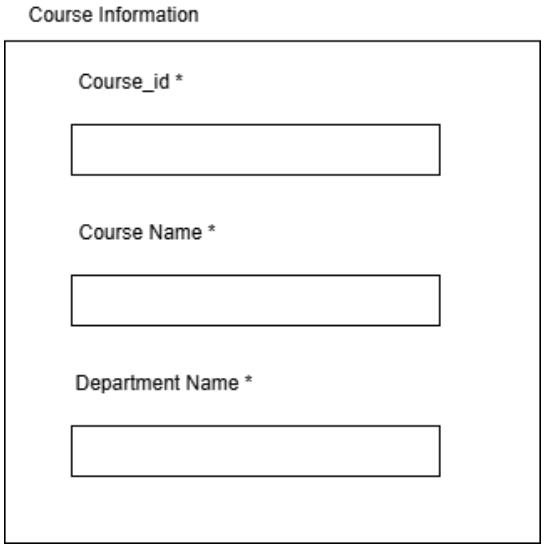
Entity Relationship (ER) Diagram

The Entity-Relationship (ER) model represents the interaction between three key entities: Student, Course, and Admission. A Student provides personal and academic details, which are linked to the Admission process. The Admission entity connects students to their chosen Course, ensuring eligibility and enrollment. The relationship allows a Student to apply for multiple Courses, while each Course can enroll multiple Students. This model ensures efficient management of data and processes in an academic admission system.


student_id(PK)
name
dob
gender
students
Register
courses
student_id(PK)
course_id(PK)
course_name
department_name
contact number
email
Enroll
Enroll
student_id(FK)
enrollments
admission date
admission_id(PK)
course_id
marksheet provider
Tc provider

3.3 GRAPHICAL USER INTERFACE (GUI)





Admission Information

Admission_id *

TC Provider *

Marksheet Provider *

Admission Date *

SUBMIT

CHAPTER 4

PROJECT DESCRIPTION

4.1 Description

The Automating Data Migration of Student Records project is aimed at reducing manual efforts, increasing efficiency, and eliminating data entry errors by automating the process of migrating student records from Excel to a MySQL database.

Educational institutions often rely on spreadsheets to store student records, leading to challenges like data inconsistency, human errors, and inefficiencies in handling large datasets. This project provides a robust, automated solution by integrating Python, Pandas, Selenium, PHP, and MySQL to manage student records effectively.

The system performs data extraction, validation, transformation, and storage while also integrating web form automation to streamline online admissions. It ensures error-free and structured data migration, making it a reliable tool for schools, colleges, and universities.

Key benefits of this system

- Automated data extraction and migration from Excel to MySQL.
- Efficient data validation to prevent errors and inconsistencies.
- Selenium-based web automation to populate online forms automatically.
- Real-time logging and email notifications for system tracking.
- Security measures to prevent unauthorized access and data corruption.

This system replaces traditional manual data entry with a fully automated and intelligent workflow, ensuring better accuracy, speed, and usability for institutions handling student records.

4.2 Key Module

1. Data Extraction and Validation

- Excel File Processing:
  - Uses Pandas to extract student records from structured or semi-structured Excel sheets.
  - Handles large datasets efficiently without manual intervention.
- Data Validation:
  - Ensures student details (Name, Age, Email, Phone, etc.) follow predefined rules.
  - Uses regular expressions (Regex) for email and phone number validation.
  - Identifies and flags missing or incorrect entries.
  - Prevents duplicate records before inserting data into the MySQL database.

2. Database Integration and Management

- MySQL Database Storage:
  - Stores student records in a structured relational database format.
  - Maintains data integrity and security to avoid loss or corruption.
- Error Handling and Duplicate Checking:
  - Prevents redundant entries using unique constraints in MySQL.
  - Logs invalid records separately for review and correction.
- Performance Optimization:
  - Uses bulk insert operations to process large volumes of student data efficiently.

3. Web Form Automation (Selenium Integration)

- Automated Form Submission:
  - Uses Selenium WebDriver to open web portals and fill student data into online forms.
  - Reduces manual work and human errors during data entry.
- Handling Dynamic Web Elements:
  - Detects form fields dynamically and enters the corresponding student details.
  - Uses explicit waits and error handling to manage slow-loading pages.
- Multi-Page Navigation:
  - Automates multi-step forms by clicking Next/Submit buttons programmatically.

4. Error Logging and Notifications

- Real-time Logging:
  - Logs data processing stages in a separate log file for debugging.
  - Records validation errors, duplicate entries, and unsuccessful migrations.
- Email Notifications:
  - Sends success/failure reports to the admin upon completion of data migration.
  - Notifies users in case of critical errors or system failures.

5. Security and Authentication (Future Enhancement)

- Role-Based Access Control:
  - Implements admin and user authentication for system security.
- Data Encryption:
  - Encrypts sensitive student data before storing it in the database.
- Audit Logging:
  - Keeps a record of data migration activities for tracking and accountability.

CHAPTER 5

SYSTEM DEVELOPMENT

5.1 LANGUAGE/TOOL

HTML (Hypertext Markup Language)

HTML serves as the structural foundation of the Student Data Management Form, providing the framework for user interaction. It ensures that students and administrators can efficiently enter and manage student records.

1. Structural Foundation

HTML defines the structure of the Student Data Management Form, organizing elements such as student details, course information, and admission records. The form layout enhances usability by grouping related fields logically, making data entry more intuitive.

2. Semantic Markup

The form utilizes semantic HTML elements like <fieldset>, <legend>, <label>, and <input> to improve structure and readability. These elements provide better organization of student data and enhance the accessibility of the form.

3. Cross-Platform Compatibility

As a web-based form, HTML ensures compatibility across all modern browsers and devices. Students and administrators can access the form from desktops, tablets, or mobile phones without rendering issues, ensuring seamless data management.

4. Accessibility

The Student Data Management Form is designed with a structured layout, making it easy for users to navigate and interact with various sections. Proper use of form elements, labels, and required fields helps ensure a smooth user experience, especially for individuals using assistive technologies.

5. Ease of Learning and Use

With its simple syntax, HTML allows for quick development and easy modifications to the form. It enables administrators to add, edit, or remove fields based on evolving requirements, making the system scalable and adaptable.

CSS (Cascading Style Sheets)

CSS is responsible for the visual presentation of the Student Data Management Form, ensuring a professional, user-friendly interface.

1. Presentation Layer

CSS styles the form with a clean and structured design, using color schemes and spacing to highlight key sections like student information, course details, and admission records. The skyblue background enhances readability while maintaining a professional look.

2. Responsive Design

The form is accessible across multiple devices, from desktops to smartphones. CSS frameworks like Bootstrap and media queries ensure the layout adjusts dynamically to different screen sizes, providing an optimal user experience.

3. Modularity and Reusability

By structuring CSS styles into reusable components, the form remains maintainable and scalable. Common styling patterns, such as input field borders and button designs, ensure consistency across all sections of the form.

4. Browser Compatibility

CSS follows best practices, including vendor prefixes and resets, to ensure a consistent experience across major browsers like Chrome, Firefox, Edge, and Safari. This prevents inconsistencies in form appearance and behavior.

5.1.1 BACKEND DESCRIPTION

The Student Data Management System is powered by PHP, a robust and widely-used server-side scripting language, handling all backend operations. The system efficiently processes student records, course details, and enrollment data while ensuring smooth interaction with a MySQL database.

1. Efficient Server-Side Processing

PHP is a lightweight and scalable scripting language that executes seamlessly on web servers. It processes form submissions efficiently, validating and inserting student data into the database without performance bottlenecks.

2. Secure Database Connectivity

The backend establishes a connection to MySQL using the mysqli extension. Secure handling of user input ensures that student records, course details, and admission data are stored reliably while minimizing potential SQL injection risks.

3. Modular Data Insertion

The system follows a structured data insertion process:

- Student details are first inserted into the students table.
- Course information is then linked to the respective student in the courses table.
- Enrollment details are stored in the enrollments table, ensuring a well-organized relational database structure.

4. Error Handling and Debugging

Error messages are displayed when data insertion fails, providing clear debugging information. The system checks for connection failures, invalid data inputs, and query execution errors, allowing for quick troubleshooting.

5. Cross-Platform Compatibility

As a web-based system, PHP runs on multiple operating systems, including Windows, Linux, and macOS. This ensures flexibility in deployment and easy integration with various hosting environments.

6. Dynamic Form Handling

The backend script captures form data using the $_POST method and processes student records dynamically. It automatically converts enrollment dates into the correct MySQL format (YYYY-MM-DD), ensuring consistency in stored records.

7. Data Consistency and Integrity

By linking student, course, and enrollment records, the system maintains data integrity. Each student's enrollment status is accurately recorded, preventing inconsistencies in the database.

5.1.2 DATABASE DESCRIPTION

MySQL

The database for Automating Data Migration of Student Records is built using MySQL, providing a structured, efficient, and scalable approach to storing student information, course details, and enrollment records.

1. Reliability and Data Integrity

- Ensures accurate and structured data storage, preventing errors and duplication.
- ACID compliance guarantees data consistency and integrity, even during failures.

2. Efficient Data Management

- Stores student details such as name, age, contact number, email, and enrollment details.
- Prevents redundant entries and automatically validates data integrity before insertion.

3. Scalability and Performance

- Optimized queries, indexing, and caching allow the system to process thousands of student records efficiently.
- Can handle bulk data uploads for large-scale educational institutions.

4. Secure Data Handling

- Role-based authentication ensures that only authorized users can access student records.
- Prevents SQL injection attacks by using parameterized queries.

5. Structured Data Organization

The database consists of three primary tables:

Table Name	Description
students	Stores student details, including student_id, name, age, gender, contact_number, and email.
courses	Manages course details, including course_id, course_title, and department_name.
enrollments	Tracks student enrollments, including enrollment_id, student_id, course_id, tc_provided, marksheet_provided, and enrollment_date.

These structured tables ensure efficient data retrieval and seamless integration with the automation system.

6. Logging and Error Tracking

- The system logs all invalid data entries and tracks errors encountered during migration.
- Validation logs allow administrators to review and correct data inconsistencies.

5.1.3 Tools Used

1. PyCharm (for Python Development)

PyCharm is an Integrated Development Environment (IDE) used for developing Python scripts in this project.

Key Features:

- Code Completion & Debugging – Helps detect and fix errors quickly.
- Selenium WebDriver Integration – Supports browser automation for form submissions.
- MySQL Connector Support – Enables database interaction within Python scripts.

Project Configuration in PyCharm:

1. Install Required Libraries
2. Load Student Data from Excel using Pandas.
3. Validate and Clean Data before inserting it into MySQL.
4. Automate Web Form Submission using Selenium.
5. Generate Logs and Email Notifications after completion.

2. XAMPP (for MySQL Database Management)

XAMPP provides a local server environment to run MySQL, allowing seamless database interactions for storing student records.

Configuration Steps:

1. Install XAMPP from Apache.
2. Start MySQL Database from the XAMPP Control Panel.
3. Create the student_data database with structured tables.
4. Manage student records using phpMyAdmin (http://localhost/phpmyadmin/).

3. Selenium WebDriver (for Web Form Automation)

Selenium is used to automate the submission of student details into a web-based form, reducing manual work.

Features Used in the Project:

- Locate web form elements using By.ID, By.NAME.
- Auto-fill input fields with student details.
- Handle dropdown selections for gender, TC provided, and marksheet status.
- Submit the form automatically after all fields are filled.

4. Postman

Postman is used to test database queries and validate API interactions in the project.

Usage in the Project:

- Testing MySQL data retrieval via PHP scripts.
- Verifying web form submissions by sending test inputs.
- Ensuring database updates work correctly when new records are added.

XAMPP

XAMPP provides a local server environment with Apache, MySQL, and PHP, used in AUTOMATING DATA MIGRATION OF STUDENT RECORD for database management.

Configurations for XAMPP in Automating Data Migration Of Student Record

1. **Download and Install XAMPP**
    - Download from Apache Friends.
    - Run the installer and install Apache and MySQL (PHP was not required in this project).
2. **Start XAMPP Services**
    - Open XAMPP Control Panel and start Apache and MySQL.
3. **Verify XAMPP Installation**
    - Navigate to http://localhost/phpmyadmin/ to manage the database.
4. **Database Configuration**
    - Created the student_data database.
    - Configured tables:
        - **students** (to store students details).
        - **courses** (to store courses details).
        - **enrollments** (to store enrollments details).

5. Logging and Email Notification System

The system includes real-time logging and email notifications to track errors and successful migrations.

Logging Mechanism:

- Stores validation failures, duplicate records, and database errors in log files.
- Helps administrators identify and resolve data issues efficiently.

Email Notification Process:

- Sends email alerts upon successful student data migration.
- Notifies admins in case of errors or system failures.

5.2 PSEUDO CODE

PYTHON CODE

```
<?php

// Database connection details

$host = "localhost";

$username = "root";  // Your MySQL username

$password = "1234";      // Your MySQL password

$dbname = "student_data"; // The database name



// Create a connection

$conn = new mysqli($host, $username, $password, $dbname);
```

```
// Check for connection errors

if ($conn->connect_error) {

    die("Connection failed: " . $conn->connect_error);

}

// Retrieve data from the HTML form

$student_id = $_POST['student_id'];

$name = $_POST['name'];

$age = $_POST['age'];

$gender = $_POST['gender'];

$contact_number = $_POST['contact_number'];

$email = $_POST['email'];

$course_id = $_POST['course_id'];

$course_title = $_POST['course_title'];

$department_name = $_POST['Department-Name'];



$tc_provided = $_POST['tc_provided'];

$marksheet_provided = $_POST['Marksheet_provided'];

$enrollment_date = $_POST['enrollment_date'];
```

```
// Convert date to correct format

$enrollment_date = date('Y-m-d', strtotime($enrollment_date));

// Insert data into the students table

$sql_student = "INSERT INTO students (student_id, name, age, gender,
contact_number, email)

            VALUES ('$student_id', '$name', '$age', '$gender', '$contact_number',
'$email')";

if ($conn->query($sql_student) === TRUE) {

   echo "New student record created successfully.";

} else {

   echo "Error: " . $sql_student . "<br>" . $conn->error;

}



// Insert data into the courses table

$sql_course = "INSERT INTO courses (course_id,student_id, course_title,
department_name)

            VALUES ('$course_id','$student_id', '$course_title', '$department_name')";

if ($conn->query($sql_course) === TRUE) {

   echo "New course record created successfully.";

} else {
```

```
    echo "Error: " . $sql_course . "<br>" . $conn->error;

}

// Insert data into the enrollments table

$sql_enrollment = "INSERT INTO enrollments (student_id, course_id, tc_provided,
marksheet_provided, enrollment_date)

                    VALUES ('$student_id', '$course_id', '$tc_provided',
'$marksheet_provided', '$enrollment_date')";

if ($conn->query($sql_enrollment) === TRUE) {

    echo "Enrollment record created successfully.";

} else {

    echo "Error: " . $sql_enrollment . "<br>" . $conn->error;

}



// Close the connection

$conn->close();

?>
```

CHAPTER 6

SYSTEM TESTING AND VALIDATION

Overview

System testing and validation are critical phases in software development, ensuring that the system performs as expected and meets the necessary functional and non-functional requirements. This phase evaluates the entire system's performance, security, usability, and reliability under various conditions. Testing helps identify defects, security vulnerabilities, or performance bottlenecks, whereas validation ensures the system aligns with its intended purpose. The primary objectives of system testing and validation in this project are:

- Ensuring data accuracy and consistency when migrating student records.
- Validating system efficiency in processing large volumes of data.
- Assessing system integration between Python scripts, MySQL database, and the web automation process.
- Identifying and mitigating security risks, such as SQL injection and unauthorized access.
- Confirming the automation process's reliability, including log generation, error handling, and email notifications.

6.1 Types of Testing

- **Unit Testing:** Testing individual components to ensure they work as expected.
- **Integration Testing:** Verifying that multiple components work together correctly.
- **System Testing:** Assessing the system’s overall functionality.
- **Acceptance Testing:** Confirming that the system meets user requirements.
- **Performance Testing:** Evaluating system response under various loads.
- **Security Testing:** Ensuring the system is protected against vulnerabilities.

6.2 Validation Methods

- **Data Validation:** Ensures that student records meet predefined constraints (e.g., valid emails, phone numbers).
- **Database Validation:** Ensures data is correctly stored in MySQL without duplication.
- **Form Submission Validation:** Checks Selenium automation for accurate web form submission.
- **Error Handling Validation:** Confirms that incorrect records trigger logs and notifications.

6.3 Test Cases

Test Name	Description	Expected Outcome	Actual Outcome	Result
Validate Student Data	Check if student name, age, and email format are correct	Successfully validates student data	As expected	Pass
MySQL Connection	Ensure database connection is established	Successfully connects to MySQL	As expected	Pass
Duplicate Entry Check	Ensure no duplicate student records are inserted	No duplicate entries	As expected	Pass
Web Form Auto-Fill	Check if Selenium correctly auto-fills student details	Form is filled and submitted	As expected	Pass
Error Logging	Ensure incorrect data is logged properly	Errors are logged in the file	As expected	Pass

6.3.1 Test case

Test Name	Description	Expected Outcome	Actual Outcome	Result
Email Notification	Check if email is sent after migration	Email received successfully	As expected	Pass
Load Testing	Verify system performance with large datasets	Handles data smoothly	As expected	Pass
SQL Injection Protection	Check if input validation prevents SQL injections	No SQL injection possible	As expected	Pass

6.4 Performance Testing

- **Load Testing:** Ensures the system efficiently handles bulk student records.
- **Stress Testing:** Examines the system's stability under high workloads.

6.5 Security Testing

- **SQL Injection Protection:** Ensures input validation prevents malicious SQL commands.
- **Data Privacy Compliance:** Confirms that student data is securely handled.

6.6 Acceptance Testing

- Validates the system against real-world student records.
- Confirms user satisfaction and functionality correctness.

6.7 Validation Output:

Automation.py | logs - 2025-04-01_18-35-56.log | testing.py | test.py

```
2025-04-01 18:35:59,474 - INFO - Starting to enter the  details of John Smith2
2025-04-01 18:35:59,474 - ERROR - Exception occured INVALID name John Smith2given by student
2025-04-01 18:35:59,475 - INFO - Starting to enter the  details of Emma Johnson
2025-04-01 18:35:59,475 - ERROR - Exception occured Invalid Age given by student :Emma Johnson
2025-04-01 18:35:59,475 - INFO - Starting to enter the  details of Alex Brown
2025-04-01 18:35:59,475 - ERROR - Exception occured INVALID contact number - number do not have 10 digits given by st
2025-04-01 18:35:59,475 - INFO - Starting to enter the  details of Sophia Davis
2025-04-01 18:35:59,475 - ERROR - Exception occured INVALID contact number - number do not have 10 digits given by st
2025-04-01 18:35:59,475 - INFO - Starting to enter the  details of Liam Wilson
2025-04-01 18:35:59,475 - ERROR - Exception occured Invalid Age given by student :Liam Wilson
2025-04-01 18:35:59,475 - INFO - Starting to enter the  details of Olivia Miller
2025-04-01 18:36:03,098 - INFO - Entered the details of Student name :Olivia Miller successfully
2025-04-01 18:36:03,098 - INFO - Starting to enter the  details of Noah Taylor
2025-04-01 18:36:03,098 - ERROR - Exception occured Invalid email noah.taylorgmail.comgiven by student :Noah Taylor
2025-04-01 18:36:03,098 - INFO - Starting to enter the  details of Ava Anderson
2025-04-01 18:36:06,602 - INFO - Entered the details of Student name :Ava Anderson successfully
2025-04-01 18:36:06,602 - INFO - Starting to enter the  details of Lucas Martinez
2025-04-01 18:36:10,088 - INFO - Entered the details of Student name :Lucas Martinez successfully
2025-04-01 18:36:48,855 - INFO - Automation process started.
2025-04-01 18:36:48,855 - INFO - Extracting data from Excel.
2025-04-01 18:36:48,855 - INFO - Updating MySQL database.
2025-04-01 18:36:48,855 - INFO - Performing web automation using Selenium.
2025-04-01 18:36:48,855 - INFO - Automation process completed successfully.
2025-04-01 18:36:48,856 - INFO - Post-processing task started.
2025-04-01 18:36:48,856 - INFO - Sending email notification...
2025-04-01 18:36:48,856 - INFO - Email sent successfully.
2025-04-01 18:36:48,856 - INFO - Post-processing task completed.
```

CHAPTER 7

USER MANUAL

Introduction

The Automating Data Migration of Student Records system is designed to streamline student record management by automating the transfer of data from Excel files to a MySQL database. This system eliminates manual data entry, ensuring accuracy, efficiency, and data security.

This manual provides a step-by-step guide on how to use the system, from uploading Excel files to validating student records and monitoring data migration progress.

System Requirements

Hardware Requirements

- Processor: Intel Core i3 or higher
- RAM: Minimum 4GB (Recommended 8GB or more)
- Storage: At least 500MB of free space

Software Requirements

- Operating System: Windows 10/11, macOS, or Linux
- Python Version: 3.x (Latest version recommended)
- Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge
- Database: MySQL (XAMPP for local hosting)
- Server: Apache (via XAMPP)
- Additional Software: PyCharm IDE (for running Python scripts)

Dashboard Overview

The system consists of the following modules:

- File Upload: Allows users to upload an Excel file containing student records.
- Data Validation: Verifies that all student records meet the predefined format.
- Database Integration: Inserts valid records into the MySQL database.
- Web Form Automation: Uses Selenium to auto-fill and submit student details in online forms.
- Logs and Reports: Displays logs for errors, duplicate entries, and migration status.

Using the Data Migration System

Uploading Student Records

1. Open the Student Data Migration System.
2. Click Upload File and select the Excel file (.xlsx) containing student records.
3. Click Submit to begin processing the file.

Data Validation

1. The system automatically checks for:
    - Valid names (Only alphabets allowed).
    - Correct age range (17-35 years).
    - Valid phone numbers (10-digit numeric values).
    - Proper email formatting (e.g., example@domain.com).
2. If errors are detected, invalid records are flagged, and a log file is generated.
3. Users can download the error report to manually correct the data.

Web Form Automation

Submitting Student Records Automatically

1. Once validation is complete, click Start Automation.
2. Selenium WebDriver will open the student registration form (http://localhost/draft_HTML.html).
3. The system will:
    - Locate form fields such as name, age, gender, contact number, and email.
    - Auto-fill the form using extracted student data.
    - Submit the form automatically.
4. A confirmation message will appear upon successful submission.

Viewing Logs and Reports

1. Click on the Logs and Reports tab.
2. View the following details:
    - Total records processed.
    - Number of successful migrations.
    - Duplicate entries found and skipped.
    - Error messages and validation failures.
3. Click Download Report to save a detailed error log.

Security and Data Privacy

- Only validated data is inserted into the database.
- Unauthorized users cannot modify stored records.
- All student data is encrypted before storage.
- Logs and reports are protected and accessible only to authorized personnel.

Logging Out

- Close the application when done.
- Ensure all processes have completed before shutting down the system.

Troubleshooting

Issue	Possible Solution
Data not uploading	Check if the Excel file format is correct (.xlsx).
Validation errors	Open the error log to review invalid records and correct them.
Web form automation not working	Ensure Selenium WebDriver is properly installed and configured.
Email notifications not received	Check SMTP settings and verify that the recipient email is correct.

CHAPTER 8

SYSTEM DEPLOYMENT

System deployment is the process of setting up a software system for use. It includes steps like testing, setting up the environment, and making sure everything works correctly. Different methods, like phased or full deployment, help in a smooth transition. For a project like automating data migration, deployment ensures data is transferred correctly and securely. Testing before deployment helps avoid errors, and regular monitoring keeps the system running smoothly. A well-planned deployment makes the system efficient and reliable.

Step 1: Setting Up the Environment

- Install Python (for data processing and validation).
- Install MySQL (using XAMPP for easy management).
- Install PHP (to handle database operations and display records).
- Install necessary Python libraries like pymysql, pandas, and selenium.

Step 2: Configuring the Database

- Create a MySQL database (student_data).
- Define tables (students, courses, enrollments).
- Ensure necessary constraints and relationships are set up.

Step 3: Preparing the Excel Data

- Ensure the Excel file follows the required format.
- Validate column names and data types to match the database schema.

Step 4: Implementing Data Validation in Python

- Read the Excel file using Pandas.
- Validate phone numbers, emails, and other necessary fields.
- Filter out incorrect records and log errors.

Step 5: Automating Data Insertion Using Selenium

- Use Selenium to automate data transfer from Excel to MySQL.
- Ensure correct mapping of Excel columns to MySQL fields.
- Insert valid records into the database while skipping incorrect ones.

Step 6: PHP Integration for Database Operations

- Use PHP to connect to MySQL and retrieve data.
- Implement queries to fetch correct, incorrect, updated, and pending records.
- Display results on an HTML page without JavaScript.

Step 7: Testing the System

- Run test cases for data validation, insertion, and retrieval.
- Check for errors in database updates and display results.
- Verify system performance and efficiency.

Step 8: Deployment and Maintenance

- Deploy the system on a local server using XAMPP.
- Monitor database performance and log errors.
- Provide regular updates to enhance efficiency.

Python code

```
import smtplib
from email.message import EmailMessage
from selenium import webdriver
from selenium.webdriver.common.by import By
import pandas as pd
import time
import logging
import os
import re
from datetime import datetime
from selenium.webdriver.support.select import Select

def is_valid_contact(contact):
    return bool(re.fullmatch(r"\d+", str(contact)))

def is_valid_number(contact):
    contact = str(contact).strip() if pd.notna(contact) else ""  # Convert to string safely
    return bool(re.fullmatch(r"\d{10}", contact))

def is_valid_name(name):
    name = str(name).strip() if pd.notna(name) else ""  # Convert to string and remove
extra spaces
    return bool(re.fullmatch(r"[A-Za-z ]+", name))

def is_valid_age(age):
    try:
        age = int(age)  # Convert to integer
        return 17 <= age <= 35  # Check if within range
    except ValueError:
```

```
        return False

def is_valid_email(mail):
    mail = str(mail).strip() if pd.notna(mail) else ""  # Convert to string and remove
spaces
    pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"  # Email regex
pattern
    return bool(re.fullmatch(pattern, mail))  # Check if email matches the pattern

current_date = datetime.now()
log_dir = 'logs'
os.makedirs(log_dir, exist_ok=True)
log_file_name = "logs - "+ current_date.strftime('%Y-%m-%d_%H-%M-%S.log')
log_file_path = os.path.join(log_dir, log_file_name)
logging.basicConfig(filename=log_file_path, level=logging.INFO,
format='%(asctime)s - %(levelname)s - %(message)s')

# Load the Excel data
file_path = "C:/Users/asus/OneDrive/Desktop/Project/dummy.xlsx"
df = pd.read_excel(file_path)
df["enrollment_date"] = pd.to_datetime(df["enrollment_date"]).dt.date

# Initialize counters
total_records = len(df)
success_count = 0
fail_count = 0

# Path to your WebDriver
driver_path = 'C:/Program Files/Google/Chrome/Application/chrome.exe'

# Path to your local HTML file
```

```
html_file_path = 'http://localhost/draft HTML.html'

# Initialize the WebDriver (outside the loop)
driver = webdriver.Chrome()

try:
    for index, row in df.iterrows():
        try:
            logging.info("Starting to enter the  details of "+row["name"])
            Student_Id = row["student_id"]
            Name = row["name"]
            if is_valid_name(Name):
                print(f"Valid name - "+row["name"])
            else:
                print(f"Invalid name - "+row["name"])
                raise Exception("INVALID name " +row["name"]+ "given by student")
            Gender = row["gender"]
            Age = row["age"]
            if is_valid_age(Age):
                print(f" Valid age ")
            else:
                print(f"Invalid age")
                raise Exception("Invalid Age given by student :" + row["name"])
            ContactNumber = row["contact_number"]
            if is_valid_contact(ContactNumber):
                print(f"contact number has only numbers")
            else:
                print(f"INVALID contact number "+row["contact_number"] +"given by
student :"+row["name"])
                raise Exception("INVALID contact number"+row["contact_number"]
+"given by student :"+row["name"])
```

```
        if is_valid_number(ContactNumber):
            print(f"Valid 10-digit contact number")
        else:
            print(f"INVALID contact number given by student :" + row["name"])
            raise Exception("INVALID contact number - number do not have 10 digits
given by student :" + row["name"])
        email = row["email"]
        if is_valid_email(email):
            print(f"Valid email ")
        else:
            print(f"Invalid email "+row["email"] + "given by student :"+row["name"])
            raise Exception("Invalid email " + row["email"] + "given by student :" +
row["name"])

        Course_Id = row["course_id"]
        Course_Title = row["course_title"]
        Department = row["department_name"]
        TC_provided = row["tc_provided_status"]
        Marksheet_Provided = row["marksheet_provided_status"]
        Admission_date = str(row["enrollment_date"])  # Convert date to string

        # Open the web page
        driver.get(html_file_path)

        # Wait for the page to load
        time.sleep(2)

        # Locate form fields
        StudentId_field = driver.find_element(By.ID, 'student-id')
        Name_field = driver.find_element(By.ID, "student-name")
        Age_field = driver.find_element(By.ID, "age")
```

```
gender_dropdown = driver.find_element(By.ID, "gender")
gender_select = Select(gender_dropdown)
ContactNumber_field = driver.find_element(By.ID, "contact-number")
Email_field = driver.find_element(By.ID, "email")
CourseId_field = driver.find_element(By.ID, "course-id")
CourseTitle_field = driver.find_element(By.ID, "course-title")
DepartmentName_field = driver.find_element(By.ID, "Department-Name")
tc_provided_dropdown = driver.find_element(By.ID, "tc-provided")
tc_provided_select = Select(tc_provided_dropdown)
tc_provided_select.select_by_visible_text("Yes")
tc_provided_select.select_by_visible_text("No")
marksheet_provided_dropdown = driver.find_element(By.ID, "Marksheet-
provided")
marksheet_provided_select = Select(marksheet_provided_dropdown)
admission_date_field = driver.find_element(By.ID, "enrollment-date")
submit_button = driver.find_element(By.NAME, "Submit")


# Fill out the form
StudentId_field.send_keys(Student_Id)
Name_field.send_keys(Name)
Age_field.send_keys(Age)
gender_select.select_by_visible_text(Gender)
ContactNumber_field.send_keys(ContactNumber)
Email_field.send_keys(email)
CourseId_field.send_keys(Course_Id)
CourseTitle_field.send_keys(Course_Title)
DepartmentName_field.send_keys(Department)
tc_provided_dropdown.send_keys(TC_provided)
marksheet_provided_dropdown.send_keys(Marksheet_Provided)
admission_date_field.send_keys(Admission_date)
```

```
            # Submit the form
            submit_button.click()

            success_count += 1

            # Wait before moving to the next record
            time.sleep(1)
            logging.info("Entered the details of Student name :"+row["name"]+"
successfully")
        except Exception as e:
            print("Exception is:", e)
            logging.error("Exception occured "+str(e))
            fail_count += 1
except Exception as e:
        print("Exception is:", e)
        logging.error("Exception is:",e)
    # print("All records processed successfully!")

finally:

    results_file_path = "C:/xampp/htdocs/result.txt"

    # Save results to a text file
    with open(results_file_path, "w") as file:
        file.write(f"total_records={total_records}\n")
        file.write(f"success_count={success_count}\n")
        file.write(f"fail_count={fail_count}\n")
    time.sleep(5)
    driver.execute_script("window.open('http://localhost/report.php', '_blank');")
    print("Results saved to results.txt")
```

```
    time.sleep(30)

    # Send email notification after processing
    sender_email = "haarishkcb@gmail.com"
    sender_password = "dnkgammhqxdtjrza"
    recipient_email = "22ucs145@loyolacollege.edu"

    msg = EmailMessage()
    msg['Subject'] = "Automation Process Completed"
    msg['From'] = sender_email
    msg['To'] = recipient_email
    msg.set_content("Hello,\n\nThe Selenium automation process has successfully
completed.\n\nRegards,\nYour Automation System")

    try:
        # Connect to Gmail SMTP server
        with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
            server.login(sender_email, sender_password)
            server.send_message(msg)
            print("Email sent successfully!")

    except Exception as e:
        print("Failed to send email:", e)


def automation_process():
    try:
        logging.info("Automation process started.")

        # Simulating data extraction
        logging.info("Extracting data from Excel.")
```

```
        # (Your code for reading Excel)

        # Simulating database update
        logging.info("Updating MySQL database.")
        # (Your code for updating MySQL)

        # Simulating Selenium operations
        logging.info("Performing web automation using Selenium.")
        # (Your Selenium automation code)

        logging.info("Automation process completed successfully.")
        # Call the function after automation
        after_completion()
    except Exception as E:
        print("Failed to send email:", E)

def after_completion():
    logging.info("Post-processing task started.")
    # Example: Send an email notification
    send_email_notification()
    logging.info("Post-processing task completed.")

def send_email_notification():
    """Simulated function to send an email notification."""
    logging.info("Sending email notification...")
    # Your email sending logic here (e.g., using smtplib)
    logging.info("Email sent successfully.")

# Run the automation process
automation_process()
```

CHAPTER 9

CONCLUSION

The Automating Data Migration of Student Records project successfully eliminates manual data entry inefficiencies and ensures accurate, fast, and secure student record management. By integrating Python, Selenium, MySQL, and PHP, the system provides a fully automated solution for extracting, validating, and storing student records while also automating web form submissions.

Key Achievements of the Project

- Automated student data extraction from Excel, reducing human effort.
- Data validation mechanisms to ensure accuracy in student records (name, age, email, etc.).
- MySQL database integration for structured and efficient record management.
- Web form automation using Selenium, eliminating the need for manual submissions.
- Logging and error tracking to handle incorrect or duplicate records effectively.
- Email notification system for real-time process updates.

The system ensures high accuracy, efficiency, and ease of use, making it an essential tool for educational institutions. It provides real-time logs, structured data management, and automated processes, allowing institutions to streamline student record handling with minimal errors.

Through comprehensive system testing and validation, the project has demonstrated high reliability and performance in managing large student datasets. The integration of database security, automated validation, and email alerts ensures a robust and scalable solution for student data migration.

This project represents a significant advancement in automation for student record management, reducing administrative workload and enhancing data security and processing speed. It serves as a foundation for future improvements in digital student record automation.

CHAPTER 10

FUTURE ENHANCEMENTS

While the project has successfully met its core objectives, several future enhancements can be implemented to improve functionality, security, and scalability.

1. Cloud-Based Database Integration

- Migrating the system to cloud databases (e.g., AWS RDS, Google Cloud SQL) for better scalability and remote access to student records.
- Multi-user access, allowing multiple administrators to manage student data from different locations.

2. AI-Based Data Validation

- Implementing machine learning models to detect inconsistent or incorrect data patterns automatically.
- AI-driven suggestions to correct invalid student records before insertion into the database.

3. Real-Time Dashboard & Analytics

- Developing an interactive dashboard to display student migration statistics, error trends, and validation reports.
- Graphical representation of student records for better decision-making and performance tracking.

4. Role-Based Access Control (RBAC)

- Implementing multi-user authentication where different roles (e.g., admin, staff, faculty) have controlled access to records.
- Restricting sensitive database modifications to authorized personnel only.

5. Mobile App Integration

- Developing a mobile-friendly version of the system to allow administrators to manage student records from smartphones.
- Enabling push notifications for record updates, errors, or system alerts.

6. Automated Backup & Recovery System

- Implementing a daily/weekly backup mechanism to store database snapshots.
- Automated recovery features to prevent data loss in case of system failure.

7. Enhanced Security Measures

- Encrypting sensitive student data using AES or SHA encryption techniques.
- Multi-factor authentication (MFA) for additional security when accessing the system.

CHAPTER 11

BIBLIOGRAPHY

Book References

- **Martin Fowler**, Refactoring: Improving the Design of Existing Code, 2nd Edition, Addison-Wesley, 2018.
- **Robert C. Martin**, Clean Code: A Handbook of Agile Software Craftsmanship, Prentice Hall, 2008.
- **Ian Sommerville**, Software Engineering, 10th Edition, Pearson, 2015.
- **Thomas Cormen, Charles Leiserson, Ronald Rivest, Clifford Stein,** Introduction to Algorithms, 3rd Edition, MIT Press, 2009.
- **Ramez Elmasri, Shamkant B. Navathe**, Fundamentals of Database Systems, 7th Edition, Pearson, 2016.
- **Mark Lutz**, Learning Python, 5th Edition, O'Reilly Media, 2013.
- **John Walkenbach**, Excel 2019 Power Programming with VBA, Wiley, 2019

Website References

- **Python Official Documentation** -https://docs.python.org
- **Selenium Official Documentation** -https://www.selenium.dev/documentation
- **W3Schools** - https://www.w3schools.com/python/pandas/pandas_csv.asp
- **GeeksforGeeks** - https://www.geeksforgeeks.org/os-path-module-python
- **Stack Overflow** - https://stackoverflow.com/questions/tagged/selenium-webdriver
- **MySQL Official Documentation** - https://dev.mysql.com/doc
- **Pandas Official Documentation** - https://pandas.pydata.org/docs
- **XAMPP Documentation** -https://www.apachefriends.org
- **Google Chrome WebDriver** – https://chromedriver.chromium.org

APPENDIX A – DATA DICTIONARY

1.Students Table

Field	Type	Null	Key	Description
student_id	int	NO	PRI	Student id
Name	varchar(100)	NO		name
Age	int	NO		age
Gender	enum('Male', 'Female')	NO		gender
contact_number	varchar(15)	YES		Contact number
Email	varchar(100)	NO		email

2. Courses Table

Field	Type	Null	Key	Description
course_id	int	NO	PRI	Course id
student_id	int	NO	PRI	Student id
course_title	varchar(100)	NO		Course title
department_name	varchar(100)	NO		Department name

3.Enrollments Table

Field	**Type**	**Null**	**Key**	**Description**
enrollment_id	Int	NO	PRI	Enrolment id
student_id	Int	NO	MUL	Student id
course_id	Int	NO		Course id
tc_provided	enum('yes','no')	NO		Tc provider
marksheet_provided	enum('yes','no')	NO		Marksheet provider
enrollment_date	Date	NO		Enrolment date

APPENDIX B – SAMPLE OUTPUT

Student Data Management Form

Student Information

Student ID *

1001

Name *

mike

Age *

19

Gender *

Male

Contact Number *

4536789101

Email *

mike123@gmail.com

Course Information

Course ID *

1200

Course Title *

b.sc

Deparment Name *

computer science

Admission Information

TC Provided *

Yes

Marksheet Provided *

Yes

Admission Date *

30-05-2022

Submit

Automation Results

Total Records Processed: 9

Successful Inserts: 3

Failed Inserts: 6